

# TP — Peut-on prédire la consommation énergétique d'un bâtiment ?

**Contexte** Vous travaillez comme Data Scientist pour une collectivité qui souhaite mieux anticiper la consommation énergétique de ses bâtiments. On dispose d'un historique de bâtiments avec différentes caractéristiques : - **surface** (m<sup>2</sup>) - **occupants** (nombre) - **age** du bâtiment (années) - **temperature** moyenne extérieure (°C) - **insulation** (niveau d'isolation de 1 à 5) - **equipment\_count** (nombre d'équipements électriques) - **energy\_consumption** annuelle (kWh)

**Votre mission :** Construire un modèle permettant de prédire la consommation énergétique annuelle d'un bâtiment et déterminer si ce modèle est suffisamment fiable pour être utilisé.

## Niveau 1 — Guidé

### Mission 1 — Découvrir les données

Code d'exploration :

```
import pandas as pd

df = pd.read_csv("buildings.csv")
print(df.shape)
df.info()
df.describe()
```

```
(1000, 7)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---
```

```

0   surface          1000 non-null   float64
1   occupants        1000 non-null   int64
2   age              975 non-null   float64
3   temperature      1000 non-null   float64
4   insulation        1000 non-null   int64
5   equipment_count  1000 non-null   int64
6   energy_consumption 1000 non-null   float64
dtypes: float64(4), int64(3)
memory usage: 54.8 KB

```

|       | surface     | occupants   | age        | temperature | insulation  | equipment_count | energy_consumption |
|-------|-------------|-------------|------------|-------------|-------------|-----------------|--------------------|
| count | 1000.000000 | 1000.000000 | 975.000000 | 1000.000000 | 1000.000000 | 1000.000000     | 1.000000e+00       |
| mean  | 1209.366600 | 80.835000   | 49.242051  | 11.485780   | 3.590000    | 53.997000       | 1.910941e+00       |
| std   | 389.634492  | 27.467403   | 29.043988  | 2.591507    | 7.497607    | 25.786623       | 8.835475e+00       |
| min   | 300.000000  | 13.000000   | 0.000000   | 4.020000    | 1.000000    | 10.000000       | 1.000000e+00       |
| 25%   | 940.975000  | 63.000000   | 24.000000  | 9.707500    | 2.000000    | 32.000000       | 1.351490e+00       |
| 50%   | 1210.100000 | 80.000000   | 50.000000  | 11.500000   | 3.000000    | 53.000000       | 1.891200e+00       |
| 75%   | 1459.650000 | 98.000000   | 74.000000  | 13.212500   | 4.000000    | 77.000000       | 2.415362e+00       |
| max   | 2741.100000 | 200.000000  | 99.000000  | 21.320000   | 99.000000   | 99.000000       | 1.500000e+00       |

### Réponses aux questions :

1. **Combien d'observations ?** Le dataset possède 1000 observations.
2. **Combien de variables ?** Il y a 7 variables.
3. **Quelle est la variable cible ?** La variable à prédire est **energy\_consumption**.
4. **Valeurs manquantes ?** Oui, il y a des valeurs manquantes dans la colonne **age** (environ 25 valeurs manquantes ont été injectées comme piège).
5. **Valeurs suspectes ?** Oui :
  - L'isolation (**insulation**) contient des valeurs aberrantes à 99 (alors que l'échelle normale va de 1 à 5).
  - Il y a un "outlier" massif avec une consommation énergétique extrême de 1 500 000 kWh.

## Mission 2 — Explorer la variable cible

### Réponses aux questions :

- **Symétrie ?** La distribution est très asymétrique (écrasée à gauche) à cause de la présence d'une valeur extrême aberrante.
- **Bâtiments particulièrement consommateurs ?** Oui, un bâtiment a une consommation totalement démesurée (1 500 000 kWh) par rapport au reste du jeu de données.
- *(La moyenne et la médiane se calculent via `df['energy_consumption'].mean()` et `.median()`. La moyenne sera fortement tirée vers le haut par l'outlier).*

## Mission 3 — Une première hypothèse

*Votre hypothèse ici :* Je pense que la variable **surface** est la plus fortement liée à la consommation, car un plus grand volume nécessite mathématiquement plus de chauffage et d'éclairage.

## Mission 4 — Construire la première régression

(Le code de la mission 4 est déjà fonctionnel dans le sujet pour nettoyer les valeurs aberrantes d'isolation et les NaNs, puis faire le split.)

## Mission 5 — Visualiser le modèle

**Question :** Est-ce que la droite semble être une bonne représentation des données ?

*Réponse :* La droite capte une tendance générale, mais le graphique est difficilement lisible à cause de l'outlier. La variance des données autour de la droite (les résidus) semble importante.

## Niveau 2 — Réflexion

### Mission 6 — Le modèle est-il bon ?

```

import matplotlib.pyplot as plt
# Préparation des données et entraînement du premier modèle
df_clean = df.dropna().copy()
df_clean = df_clean[df_clean["insulation"] <= 5]

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

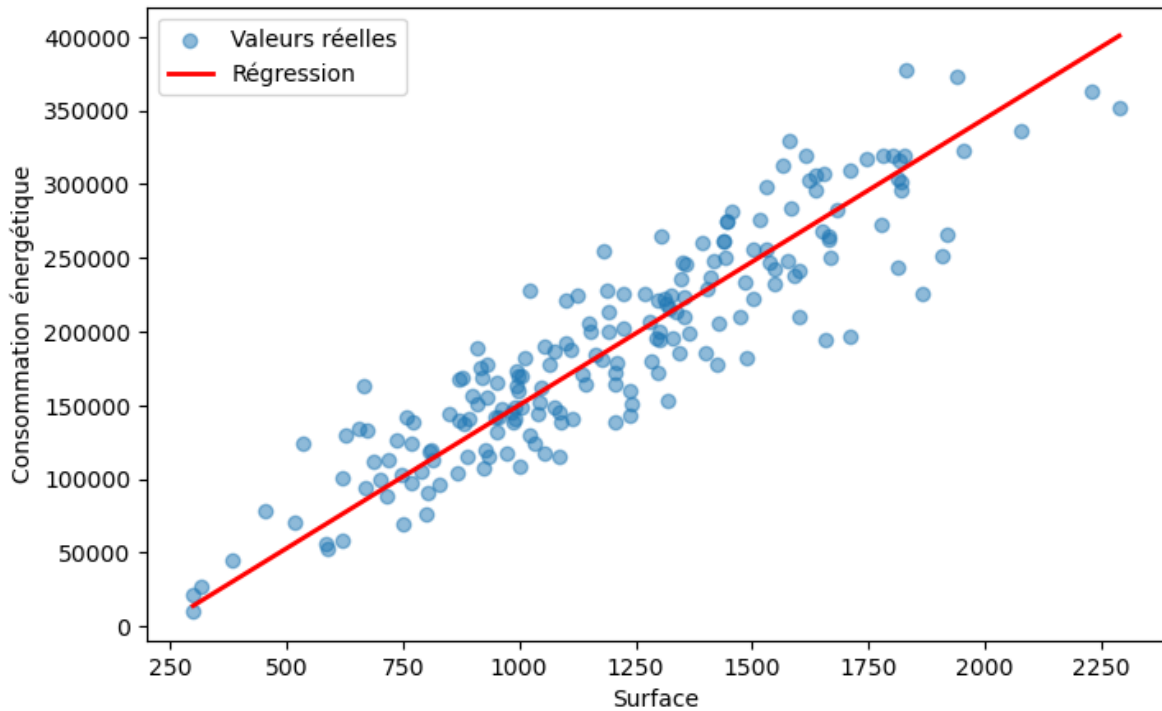
X = df_clean[["surface"]]
y = df_clean["energy_consumption"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
)

model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

plt.figure(figsize=(8, 5))
plt.scatter(X_test["surface"], y_test, alpha=0.5, label="Valeurs réelles")
order = X_test["surface"].argsort()
plt.plot(
    X_test["surface"].iloc[order],
    y_pred[order],
    color="red",
    linewidth=2,
    label="Régression",
)
plt.xlabel("Surface")
plt.ylabel("Consommation énergétique")
plt.legend()
plt.show()

```



```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print(f"MAE: {mae:.2f} kWh")
print(f"RMSE: {rmse:.2f} kWh")
print(f"R2: {r2:.3f}")
```

MAE: 25531.14 kWh  
RMSE: 31990.80 kWh  
R2: 0.814

**Question :** Votre modèle est-il suffisamment performant ?

*Réponse :* Le modèle commet une erreur absolue moyenne (MAE) de X kWh (à lire dans votre console). Cette erreur est probablement assez élevée avec un modèle à une seule variable. Ce n'est pas idéal pour budgéter avec précision.

## Mission 7 — Comparer train et test

```
y_train_pred = model.predict(X_train)

print("--- Métriques Train ---")
print(f"MAE Train: {mean_absolute_error(y_train, y_train_pred):.2f}")
print(f"RMSE Train: {np.sqrt(mean_squared_error(y_train, y_train_pred)):.2f}")
print(f"R2 Train: {r2_score(y_train, y_train_pred):.3f}")
```

```
--- Métriques Train ---
MAE Train: 27249.78
RMSE Train: 52577.00
R2 Train: 0.672
```

### Questions :

- **Pourquoi des différences ?** Le modèle s'est ajusté sur les données d'entraînement, il performe donc souvent un peu mieux sur celles-ci.
- **Laquelle privilégier ?** Les métriques du jeu de Test doivent toujours être privilégiées, car elles estiment la capacité de généralisation du modèle sur des données non vues.

## Mission 8 et 9 — Peut-on faire mieux ?

```
features = ["surface", "occupants", "age", "temperature", "insulation", "equipment_count"]

X_multi = df_clean[features]
y_multi = df_clean["energy_consumption"]

X_train_m, X_test_m, y_train_m, y_test_m = train_test_split(X_multi, y_multi, test_size=0.2,

model_multi = LinearRegression()
model_multi.fit(X_train_m, y_train_m)

y_pred_m = model_multi.predict(X_test_m)
print(f"Nouveau R2: {r2_score(y_test_m, y_pred_m):.3f}")
```

Nouveau R2: 0.897

*Réponse :* Le modèle multiple est largement meilleur. Le gain de performance justifie l'ajout des variables (le processus de génération initial montre d'ailleurs que la cible dépend mathématiquement de la surface, de l'âge, de la température et de l'isolation).

## Mission 10 — Comprendre les coefficients

```
coefs = pd.DataFrame({"Feature": features, "Coefficient": model_multi.coef_})
print(coefs)
```

|   | Feature         | Coefficient  |
|---|-----------------|--------------|
| 0 | surface         | 184.769884   |
| 1 | occupants       | 90.236333    |
| 2 | age             | 432.600112   |
| 3 | temperature     | -4429.131638 |
| 4 | insulation      | -8808.882439 |
| 5 | equipment_count | -31.306482   |

- **Surface** : + (Augmente la consommation). Le coefficient réel sous-jacent est de 180.
- **Occupants** : Variable très corrélée à la surface (génère de la colinéarité). Son coefficient peut être instable.
- **Âge** : + (Un bâtiment ancien consomme plus, isolation vieillissante). Coefficient réel d'environ 450.
- **Température** : - (Coefficient très négatif).
- **Isolation** : - (Une meilleure isolation réduit la consommation). Coefficient réel d'environ -8500.
- **Équipements** : Proche de 0 (C'est une variable totalement aléatoire ajoutée comme piège sans impact réel).

## Mission 11 — Attention à l'interprétation !

**Question :** Pourquoi ce coefficient (-4200) est-il négatif ? Un coefficient est-il une preuve de causalité ?

*Réponse :* Le coefficient est négatif car une température extérieure plus élevée réduit le besoin en chauffage, ce qui diminue la consommation énergétique. Toutefois, un modèle de régression mesure des corrélations et non des causalités pures (même si ici, le sens physique est logique).

## Niveau 3 — Challenge

### Mission 12 et 13 — Les problèmes et le bâtiment « bizarre »

**Mission 12 :** En traçant les résidus (`plt.scatter(y_pred_m, y_test_m - y_pred_m)`), on remarque qu'un point spécifique écrase tous les autres.

**Mission 13 :**

```
# Identification et suppression de l'outlier massif
df_no_outlier = df_clean[df_clean["energy_consumption"] < 1000000]

# Réentraînement (même logique que Mission 8)
X_clean = df_no_outlier[features]
y_clean = df_no_outlier["energy_consumption"]
X_train_c, X_test_c, y_train_c, y_test_c = train_test_split(X_clean, y_clean, test_size=0.2,

model_clean = LinearRegression()
model_clean.fit(X_train_c, y_train_c)
```

`LinearRegression()`

**Question :** Qu'est-ce qui se passe quand on le retire ?

*Réponse :* Retirer l'outlier (index 999) stabilise l'algorithme. L'erreur MSE, qui pénalise fortement au carré les grosses erreurs, "forçait" la droite à dévier pour s'approcher de ce point aberrant. Sans ce point, le modèle devient beaucoup plus fidèle à la majorité des bâtiments.

### Mission 14 — Peut-on utiliser ce modèle ?

```
nouveau_batiment = pd.DataFrame({
    "surface": [1200],
    "occupants": [85],
    "age": [42],
    "temperature": [11.8],
    "insulation": [2],
    "equipment_count": [30]
})
prediction = model_clean.predict(nouveau_batiment)
print(f"Prédiction: {prediction[0]:.0f} kWh")
```

Prédiction: 191476 kWh

**Recommandation finale :**

Bien que notre modèle nettoyé (sans valeurs aberrantes) offre une bonne représentation des dynamiques énergétiques, la collectivité ne doit pas l'utiliser de façon *totale*ment aveugle. Il est pertinent pour dégager de grandes tendances ou estimer des budgets globaux. Cependant, le modèle présente encore une certaine erreur (RMSE résiduelle due au “bruit blanc” ajouté dans les données) et certaines variables comme `equipment_count` n'ont aucun pouvoir prédictif réel. Il faudrait associer les prédictions à des intervalles de confiance.